

Solutions — Part 1 (Abstract Classes)

- Below are the full solutions for the abstract class practice tasks.

Solution 1 — Vehicle / Car / Bike

- Abstract Vehicle class with required abstract methods.
- Concrete Car and Bike implement the required behavior.

```
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def start_engine(self):
        pass

    @abstractmethod
    def get_max_speed(self):
        pass

class Car(Vehicle):
    def start_engine(self):
        return "Car engine started with key."

    def get_max_speed(self):
        return 220

class Bike(Vehicle):
    def start_engine(self):
        return "Bike engine started with self-start."

    def get_max_speed(self):
        return 140

c = Car()
b = Bike()

print(c.start_engine(), c.get_max_speed())
print(b.start_engine(), b.get_max_speed())
```

Solution 2 — Report / PDFReport / ExcelReport

- Abstract report class with two concrete outputs.

```
from abc import ABC, abstractmethod

class Report(ABC):
    @abstractmethod
    def generate(self):
        pass

class PDFReport(Report):
    def generate(self):
        return "PDF Report generated."

class ExcelReport(Report):
    def generate(self):
        return "Excel Report generated."

r1 = PDFReport()
r2 = ExcelReport()

print(r1.generate())
print(r2.generate())
```

Solutions — Part 2 (Exception Handling)

- Below are the full solutions for the exception handling tasks.

Solution 3 — Safe Calculator

- Handles invalid input and division errors.

```
def safe_calculator():
    try:
        a = float(input("Enter first number: "))
        b = float(input("Enter second number: "))
        op = input("Enter operator (+, -, *, /): ")

        if op == '+': return a + b
        elif op == '-': return a - b
        elif op == '*': return a * b
        elif op == '/':
            try:
                return a / b
            except ZeroDivisionError:
                return "Error: Cannot divide by zero."
        else:
            return "Invalid operator."
    except ValueError:
        return "Invalid number input."

print(safe_calculator())
```

Solution 4 — Login System

- Handles missing user and wrong password.

```
users = {"ali": "1234", "sara": "abcd"}
def login(username, password):
    try:
        if username not in users:
            raise KeyError("User not found.")
        if users[username] != password:
            raise ValueError("Incorrect password.")
    except KeyError as e:
        return str(e)
    except ValueError as e:
        return str(e)
    else:
        return "Login successful!"

print(login("ali", "1234"))
print(login("ali", "wrong"))
print(login("missing", "123"))
```

Solution 5 — Safe File Reader

- Uses try/except/finally for safe file access.

```
def read_file(path):  
    try:  
        f = open(path, 'r')  
    except FileNotFoundError:  
        return "Error: File not found."  
    else:  
        data = f.read()  
        return data  
    finally:  
        try:  
            f.close()  
        except:  
            pass  
  
print(read_file("data.txt"))
```